

宁波大学
NINGBO UNIVERSITY

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	**Linux 系统 Makefile 编写和 GCC 编译实验**

一、实验目的与要求：

1.1 实验目的

(1) 理解 Makefile 的作用和重要性，学习如何编写 Makefile 来管理复杂的编译过程。

(2) 掌握 Makefile 的基本语法和结构，能够编写 Makefile 来自动化编译任务。

(3) 掌握 GCC 编译器的使用方法，了解其基本选项和操作流程。

(4) 学习如何编译 C 语言程序，以及如何对编译过程进行控制和管理。

1.2 实验要求

(1) 学习 Makefile 中的目标（target）、依赖文件（dependency_file）和命令（command）的关系。

(2) 编写 Makefile 来编译一个简单的程序，包括设置编译器、编译选项和编译规则。

(3) 理解 Makefile 中的变量和模式规则，能够使用它们来简化 Makefile 的编写和维护。

(4) 准备实验环境，编写简单的 C 语言程序，并使用 GCC 编译器进行编译。

(5) 熟悉 GCC 编译过程中的各个阶段，包括预处理、编译、汇编和链接。

二、实验设备：

(1) 华清远见开发环境。

(2) 实验平台为华清远见提供的虚拟机环境。

(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。

(4) 虚拟机工具软件为 VMware Player。

(5) FS4412 开发板。

三、实验方法（原理、流程图）

原理：

第 5 章 Linux Shell 编程实验

Shell 是 Linux 系统中用户与内核进行交互操作的接口，它的主要功能是接收用户输入的命令并将其送入内核执行。实际上，Shell 是一个命令解释器，它解释用户输入的命令并将其发送到内核执行。Shell 还具有自己的编程语言，允许用户编写由 Shell 命令组成的程序，这些程序可以与其他应用程序一样发挥作用。

Linux 系统提供了类似于 Microsoft Windows 的可视化命令输入界面，即 X Window 的图形用户界面（GUI）。在 GUI 中，用户可以使用桌面环境系统进行操作，如窗口、图标和菜单，所有的管理都可以通过鼠标进行控制。每个 Linux 系统的用户都可以拥有自己的用户界面或 Shell，以满足其特定的 Shell 需求。

在 Linux 中，有多种不同版本的 Shell，主要包括：

Bourne Shell（sh）：由贝尔实验室开发。

BASH（Bourne Again Shell）：GNU 操作系统上默认的 Shell。

Korn Shell（ksh）：对 Bourne Shell 的发展，在大部分内容上与 Bourne Shell 兼容。

C Shell（csh）：SUN 公司 Shell 的 BSD 版本。

Z Shell（zsh）：具有多种 Shell 特性的终极 Shell，集成了 bash、ksh 的特性，并增加了自己独有的特性。

无论使用哪种 Shell，其主要功能都是解释用户在命令行提示符下输入的指令。Shell 会对命令行进行语法分析，将其分解成以空白区分的符号（token），包括空白、跳格键（tab）和换行符（New Line）。如果命令包含元字符（metacharacter），Shell 将评估其正确用法。此外，Shell 还管理文件输入输出以及后台处理。在处理命令行后，Shell 会查找并执行相应的

命令。

因此，Shell 编程在 Linux 系统中具有重要的作用，用户可以通过编写 Shell 脚本来自动化任务、管理系统和进行批量处理，提高工作效率和系统管理能力。

第 6 章　　GCC 编译实验

GNU CC（简称为 Gcc）是 GNU 项目中符合 ANSI C 标准的编译系统，能够编译用 C、C++和 Object C 等语言编写的程序。gcc 不仅功能强大，而且可以编译如 C、C++、Object C、Java、Fortran、Pascal、Modula-3 和 Ada 等多种语言，而且 gcc 又是一个交叉平台编译器，它能够在当前 CPU 平台上为多种不同体系结构的硬件平台开发软件，因此尤其适合在嵌入式领域的开发编译。

下表是 gcc 支持编译源文件的后缀及其解释。

Gcc 所支持后缀名解释

后 缀 名	所对应的语言	后 缀 名	所对应的语言
.c	C 原始程序	.s/.S	汇编语言原始程序
.C/.cc/.cxx	C++原始程序	.h	预处理文件（头文件）
.m	Objective-C 原始程序	.o	目标文件
.i	已经过预处理的 C 原始程序	.a/.so	编译后的库文件
.ii	已经过预处理的 C++原始程序		

如本章开头提到的，gcc 的编译流程分为了 4 个步骤，分别为：

- 预处理（Pre-Processing）；
- 编译（Compiling）；
- 汇编（Assembling）；
- 链接（Linking）。

gcc 有超过 100 个的可用选项，主要包括总体选项、告警和出错选项、优化选项和体系结构相关选项。以下对每一类中最常用的选项进行讲解。

（1）总体选项

gcc 的总结选项如下表所示，很多在前面的示例中已经有所涉及。

gcc 总体选项列表

后　缀　名	所对应的语言
-c	只是编译不链接，生成目标文件“.o”
-S	只是编译不汇编，生成汇编代码
-E	只进行预编译，不做其他处理
-g	在可执行程序中包含标准调试信息

-o file	把输出文件输出到 file 里
-v	打印出编译器内部编译各过程的命令行信息和编译器的版本
-I dir	在头文件的搜索路径列表中添加 dir 目录
-L dir	在库文件的搜索路径列表中添加 dir 目录
-static	链接静态库
-llibrary	连接名为 library 的库文件

对于“-c”、“-E”、“-o”、“-S”选项在前一小节中已经讲解了其使用方法，在此主要讲解另外两个非常常用的库依赖选项“-I dir”和“-L dir”。

实验三：

Makefile 基本结构

Makefile 是 Make 读入的惟一配置文件，因此本节的内容实际就是讲述 Makefile 的编写规则。在一个 Makefile 中通常包含如下内容：

- 需要由 make 工具创建的目标体（target），通常是目标文件或可执行文件；
- 要创建的目标体所依赖的文件（dependency_file）；
- 创建每个目标体时需要运行的命令（command）。

它的格式为：

```
target: dependency_files
      command
```

例如，有两个文件分别为 hello.c 和 hello.h，创建的目标体为 hello.o，执行的命令为 gcc 编译指令：gcc –c hello.c，那么，对应的 Makefile 就可以写为：

```
#The simplest example
hello.o: hello.c hello.h
      gcc –c hello.c –o hello.o
```

接着就可以使用 make 了。使用 make 的格式为：make target，这样 make 就会自动读入 Makefile（也可以是首字母小写 makefile）并执行对应 target 的 command 语句，并会找到相应的依赖文件。如下所示：

```
[root@localhost makefile]# make hello.o
gcc –c hello.c –o hello.o
[root@localhost makefile]# ls
hello.c  hello.h  hello.o  Makefile
```

可以看到，Makefile 执行了“hello.o”对应的命令语句，并生成了“hello.o”目标体。

Makefile 变量

上面示例的 Makefile 在实际中是几乎不存在的，因为它过于简单，仅包含两个文件和一个命令，在这种情况下完全不必要编写 Makefile 而只需在 Shell 中直接输入即可，在实际中使用的 Makefile 往往是包含很多的文件和命令的，这也是 Makefile 产生的原因。下面就可给出稍微复杂一些的 Makefile 进行讲解：

```
sunq:kang.o yul.o
Gcc kang.o bar.o -o myprog
kang.o : kang.c kang.h head.h
Gcc –Wall –O -g –c kang.c -o kang.o
yul.o : bar.c head.h
Gcc - Wall –O -g –c yul.c -o yul.o
```

在这个 Makefile 中有 3 个目标体（target），分别为 sunq、kang.o 和 yul.o，其中第一个目标体的依赖文件就是后两个目标体。如果用户使用命令“make sunq”，则 make 管理器就是找到 sunq 目标体开始执行。

这时，make 会自动检查相关文件的时间戳。首先，在检查“kang.o”、“yul.o”和“sunq”3 个文件的时间戳之前，它会向下查找那些把“kang.o”或“yul.o”作为目标文件的时间戳。比如，“kang.o”的依赖文件为“kang.c”、“kang.h”、“head.h”。如果这些文件中任何一个的时间戳比“kang.o”新，则命令“gcc –Wall –O -g –c kang.c -o kang.o”将会执行，从而更新文件“kang.o”。在更新完“kang.o”或“yul.o”之后，make 会检查最初的“kang.o”、“yul.o”和“sunq”3 个文件，只要文件“kang.o”或“yul.o”中的任比文件时间戳比“sunq”新，则第二行命令就会被执行。这样，make 就完成了自动检查时间戳的工作，开始执行编译工作。这也就是 Make 工作的基本流程。

接下来，为了进一步简化编辑和维护 Makefile，make 允许在 Makefile 中创建和使用变量。变量是在 Makefile 中定义的名字，用来代替一个文本字符串，该文本字符串称为该变量的值。在具体要求下，这些值可以代替目标体、依赖文件、命令以及 makefile 文件中其他部分。在 Makefile 中的变量定义有两种方式：一种是递归展开方式，另一种是简单方式。

递归展开方式定义的变量是在引用在该变量时进行替换的，即如果该变量包含了对其他变量的应用，则在引用该变量时一次性将内嵌的变量全部展开，虽然这种类型的变量能够很好地完成用户的指令，但是它也有严重的缺点，如不能在变量后追加内容（因为语句：CFLAGS = $(CFLAGS) -O 在变量扩展过程中可能导致无穷循环）。

为了避免上述问题，简单扩展型变量的值在定义处展开，并且只展开一次，因此它不包含任何对其他变量的引用，从而消除变量的嵌套引用。

递归展开方式的定义格式为：VAR=var。

简单扩展方式的定义格式为：VAR：=var。

Make 中的变量使用均使用格式为：$(VAR)。

下面给出了上例中用变量替换修改后的 Makefile，这里用 OBJS 代替 kang.o 和 yul.o，用 CC 代替 gcc，用 CFLAGS 代替“-Wall -O –g”。这样在以后修改时，就可以只修改变量定义，而不需要修改下面的定义实体，从而大大简化了 Makefile 维护的工作量。

经变量替换后的 Makefile 如下所示：

```
OBJS = kang.o yul.o
CC = gcc
CFLAGS = -Wall -O -g
sunq : $(OBJS)
```

```
	$(CC) $(OBJS) -o sunq
kang.o : kang.c kang.h
	$(CC) $(CFLAGS) -c kang.c -o kang.o
yul.o : yul.c yul.h
	$(CC) $(CFLAGS) -c yul.c -o yul.o
```

可以看到，此处变量是以递归展开方式定义的。

Makefile 中的变量分为用户自定义变量、预定义变量、自动变量及环境变量。如上例中的 OBJS 就是用户自定义变量，自定义变量的值由用户自行设定，而预定义变量和自动变量为通常在 Makefile 都会出现的变量，其中部分有默认值，也就是常见的设定值，当然用户可以对其进行修改。

预定义变量包含了常见编译器、汇编器的名称及其编译选项。下表列出了 Makefile 中常见预定义变量及其部分默认值。

Makefile 中常见预定义变量

命 令 格 式	含　义
AR	库文件维护程序的名称，默认值为 ar
AS	汇编程序的名称，默认值为 as
CC	C 编译器的名称，默认值为 cc
CPP	C 预编译器的名称，默认值为$(CC) －E
CXX	C++编译器的名称，默认值为 g++
FC	FORTRAN 编译器的名称，默认值为 f77
RM	文件删除程序的名称，默认值为 rm －f
ARFLAGS	库文件维护程序的选项，无默认值
ASFLAGS	汇编程序的选项，无默认值
CFLAGS	C 编译器的选项，无默认值
CPPFLAGS	C 预编译的选项，无默认值
CXXFLAGS	C++编译器的选项，无默认值
FFLAGS	FORTRAN 编译器的选项，无默认值

可以看出，上例中的 CC 和 CFLAGS 是预定义变量，其中由于 CC 没有采用默认值，因此，需要把“CC=gcc”明确列出来。

由于常见的 gcc 编译语句中通常包含了目标文件和依赖文件，而这些文件在 Makefile

文件中目标体的一行已经有所体现，因此，为了进一步简化 Makefile 的编写，就引入了自动变量。自动变量通常可以代表编译语句中出现目标文件和依赖文件等，并且具有本地含义（即下一语句中出现的相同变量代表的是下一语句的目标文件和依赖文件）。下表列出了 Makefile 中常见自动变量。

Makefile 中常见自动变量

命 令 格 式	含　义
$*	不包含扩展名的目标文件名称
$+	所有的依赖文件，以空格分开，并以出现的先后为序，可能包含重复的依赖文件
$<	第一个依赖文件的名称
$?	所有时间戳比目标文件晚的依赖文件，并以空格分开
$@	目标文件的完整名称
$^	所有不重复的依赖文件，以空格分开
$%	如果目标是归档成员，则该变量表示目标的归档成员名称

自动变量的书写比较难记，但是在熟练了之后会非常的方便，请读者结合下例中的自动变量改写的 Makefile 进行记忆。

```
OBJS = kang.o yul.o
CC = Gcc
CFLAGS = -Wall -O -g
sunq : $(OBJS)
     $(CC) $^ -o $@
kang.o : kang.c kang.h
     $(CC) $(CFLAGS) -c $< -o $@
yul.o : yul.c yul.h
     $(CC) $(CFLAGS) -c $< -o $@
```

另外，在 Makefile 中还可以使用环境变量。使用环境变量的方法相对比较简单，make 在启动时会自动读取系统当前已经定义了的环境变量，并且会创建与之具有相同名称和数值的变量。但是，如果用户在 Makefile 中定义了相同名称的变量，那么用户自定义变量将会覆盖同名的环境变量。

Makefile 规则

Makefile 的规则是 Make 进行处理的依据，它包括了目标体、依赖文件及其之间的命令语句。一般的，Makefile 中的一条语句就是一个规则。在上面的例子中，都显示地指出了 Makefile 中的规则关系，如“$(CC) $(CFLAGS) -c $< -o $@”，但为了简化 Makefile 的编写，make 还定义了隐式规则和模式规则，下面就分别对其进行讲解。

1．隐式规则

隐含规则能够告诉 make 怎样使用传统的技术完成任务，这样，当用户使用它们时就不必详细指定编译的具体细节，而只需把目标文件列出即可。Make 会自动搜索隐式规则目录来确定如何生成目标文件。如上例就可以写成：

```
OBJS = kang.o yul.o
CC = Gcc
CFLAGS = -Wall -O -g
sunq : $(OBJS)
     $(CC) $^ -o $@
```

为什么可以省略后两句呢？因为 Make 的隐式规则指出：所有“.o”文件都可自动由“.c”文件使用命令“$(CC) $(CPPFLAGS) $(CFLAGS) -c file.c –o file.o”生成。这样“kang.o”和“yul.o”就会分别调用“$(CC) $(CFLAGS) -c kang.c -o kang.o”和“$(CC) $(CFLAGS) -c yul.c -o yul.o”生成。

下表给出了常见的隐式规则目录：

Makefile 中常见隐式规则目录

对应语言后缀名	规　　则
C 编译：**.c** 变为**.o**	$(CC) －c $(CPPFLAGS) $(CFLAGS)
C++编译：**.cc 或.C** 变为**.o**	$(CXX) -c $(CPPFLAGS) $(CXXFLAGS)
Pascal 编译：**.p** 变为**.o**	$(PC) -c $(PFLAGS)
Fortran 编译：**.r** 变为**-o**	$(FC) -c $(FFLAGS)

2．模式规则

模式规则是用来定义相同处理规则的多个文件的。它不同于隐式规则，隐式规则仅仅能够用 make 默认的变量来进行操作，而模式规则还能引入用户自定义变量，为多个文件建立相同的规则，从而简化 Makefile 的编写。

模式规则的格式类似于普通规则，这个规则中的相关文件前必须用“%”标明。使用模式规则修改后的 Makefile 的编写如下：

```
OBJS = kang.o yul.o
CC = Gcc
CFLAGS = -Wall -O -g
sunq : $(OBJS)
     $(CC) $^ -o $@
%.o : %.c
     $(CC) $(CFLAGS) -c $< -o $@
```

Make 管理器的使用

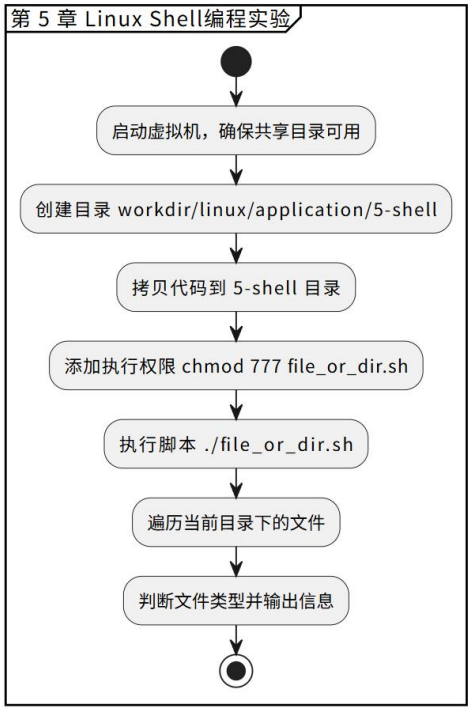
使用 Make 管理器非常简单，只需在 make 命令的后面键入目标名即可建立指定的目标，如果直接运行 make，则建立 Makefile 中的第一个目标。

此外 make 还有丰富的命令行选项，可以完成各种不同的功能。下表 3.17 列出了常用的 make 命令行选项。

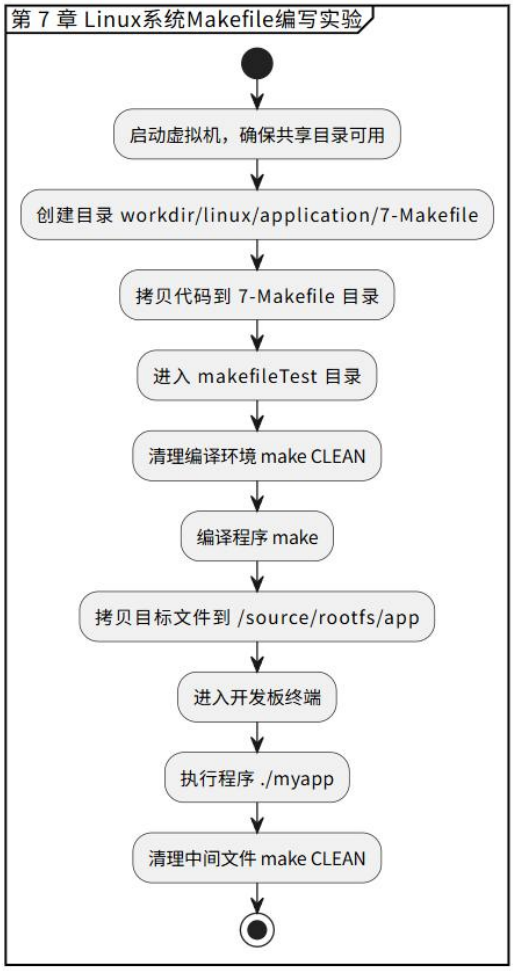
表 3.14　make 的命令行选项

命 令 格 式	含　义
-C dir	读入指定目录下的 Makefile
-f file	读入当前目录下的 file 文件作为 Makefile
-i	忽略所有的命令执行错误
-I dir	指定被包含的 Makefile 所在目录
-n	只打印要执行的命令，但不执行这些命令
-p	显示 make 变量数据库和隐含规则
-s	在执行命令时不显示命令
-w	如果 make 在执行过程中改变目录，则打印当前目录名

流程图如下



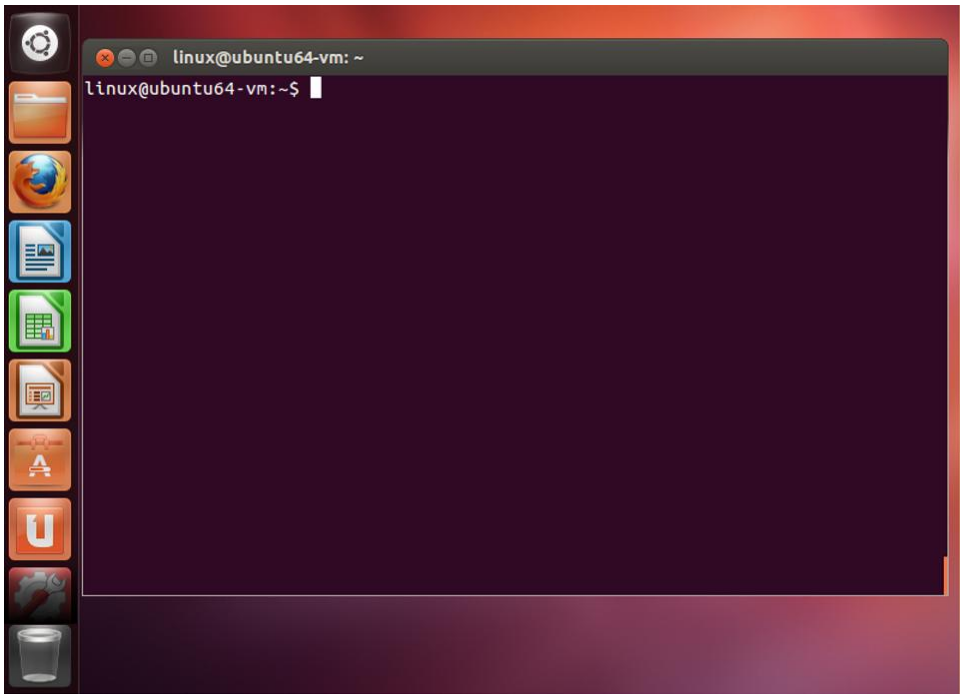




四、实验过程、步骤及内容

4.1 第 5 章实验 Linux Shell 编程

4.1.1 准备环境



4.1.2 拷贝代码

将【华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\02. Linux 系统 shell 程序设计实验\实验代码】目录拷贝到虚拟机共享目录下。

1、 建立相关目录；

```
$ mkdir workdir/linux/

$ mkdir workdir/linux/application

$ cd workdir/linux/application

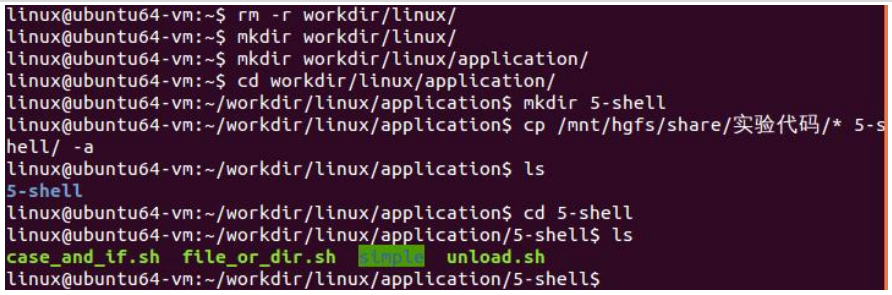
$ mkdir 5-shell
```

2、 将代码从共享目录拷入虚拟机 Linux 操作系统下；

```
$ mkdir 5-shell

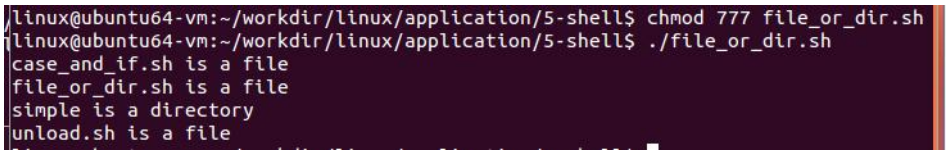
$ cp /mnt/hgfs/share/实验代码/02.\ Linux 系统 shell 程序设计实验/实验代码/* 5-shell/ -a

$ cd 5-shell/
```



4.1.3 执行代码

```
$ chmod 777 file_or_dir.sh                    //给脚本添加可执行权限
$ ./file_or_dir.sh                            //执行脚本，脚本的作用是判断该目录下的文件时目录还是普通文件
```

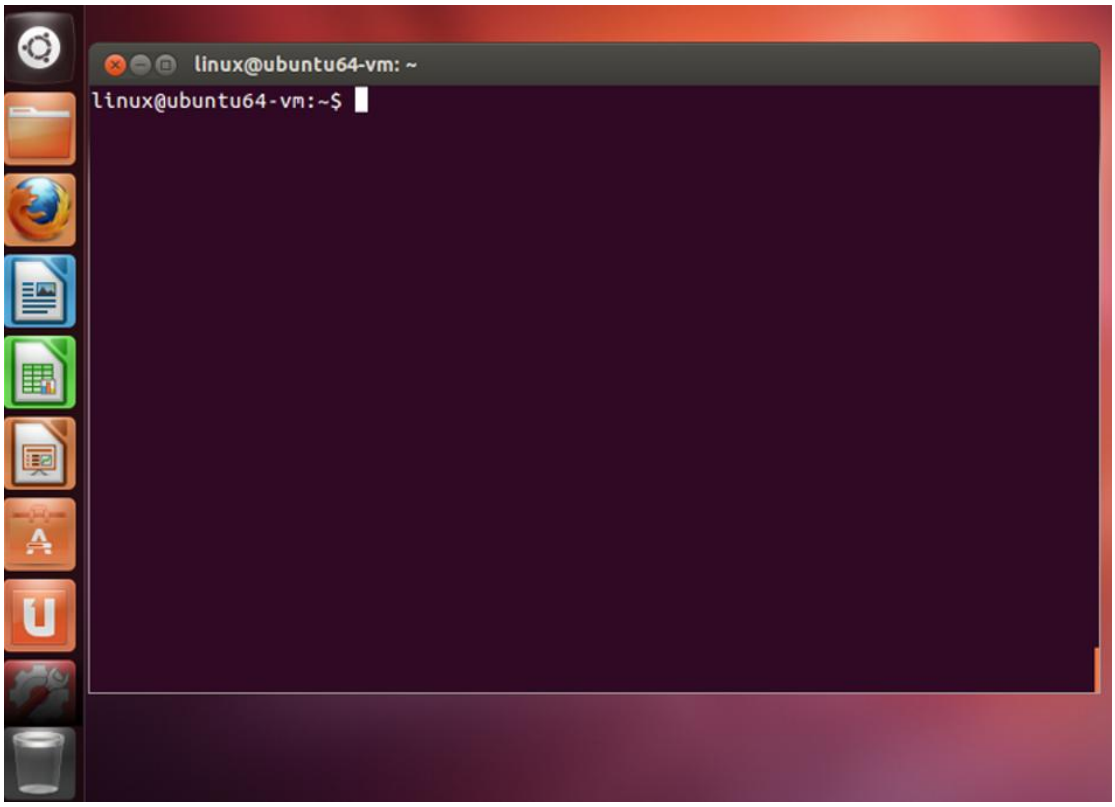


4.1.4 相关代码

```
#!/bin/sh
for i in *
do
    if [ -f $i ]
    then
        echo "$i is a file"
    elif [ -d $i ]
    then
        echo "$i is a directory"
    fi
done
```

4.2 第 6 章　GCC 编译实验

4.2.1 准备环境



4.2.2 拷贝代码

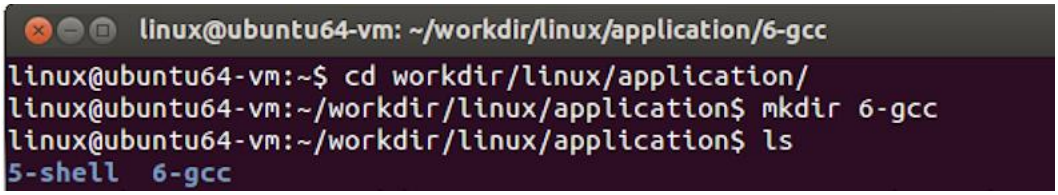
将【华清远见-CORTEXA9 资料：实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\03. Linux 系统 GCC 编译器的使用实验\实验代码】目录拷贝到虚拟机共享目录下。

将 gcc 部分代码拷贝到虚拟机 Linux 下。

4.2.3 建立相关目录；

```
$ cd workdir/linux/application

$ mkdir 6-gcc
```



4.2.4 将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【crtl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/实验代码/03.\ Linux 系统 GCC 编译器的使用实验/实验代码/* 6-gcc/ -a

$ cd 6-gcc/
```

```
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/
5/ 6/
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/6/* 6-gcc/ -a
linux@ubuntu64-vm:~/workdir/linux/application$ cd
11-pthread/ 5-shell/    6-gcc/      7-Makefile/ 8-io/       9-process/
linux@ubuntu64-vm:~/workdir/linux/application$ cd
11-pthread/ 5-shell/    6-gcc/      7-Makefile/ 8-io/       9-process/
linux@ubuntu64-vm:~/workdir/linux/application$ cd 6-gcc/
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ ls
hello  helloworld.c
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$
```

4.2.5 编译代码

```
$ arm-none-linux-gnueabi-gcc helloworld.c -o hello

$ mkdir /source/rootfs/app

$ cp hello /source/rootfs/app/
```

```
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ ls
hello  helloworld.c
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ cp hello /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ arm-none-linux-gnueabi-gcc
helloworld.c -o hello
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ mkdir /source/rootfs/app
mkdir: 无法创建目录"/source/rootfs/app": 文件已存在
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ cp hello /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$
```

4.2.5 执行代码

在开发板串口终端执行应用程序。

```
# cd /app

# ./hello
```

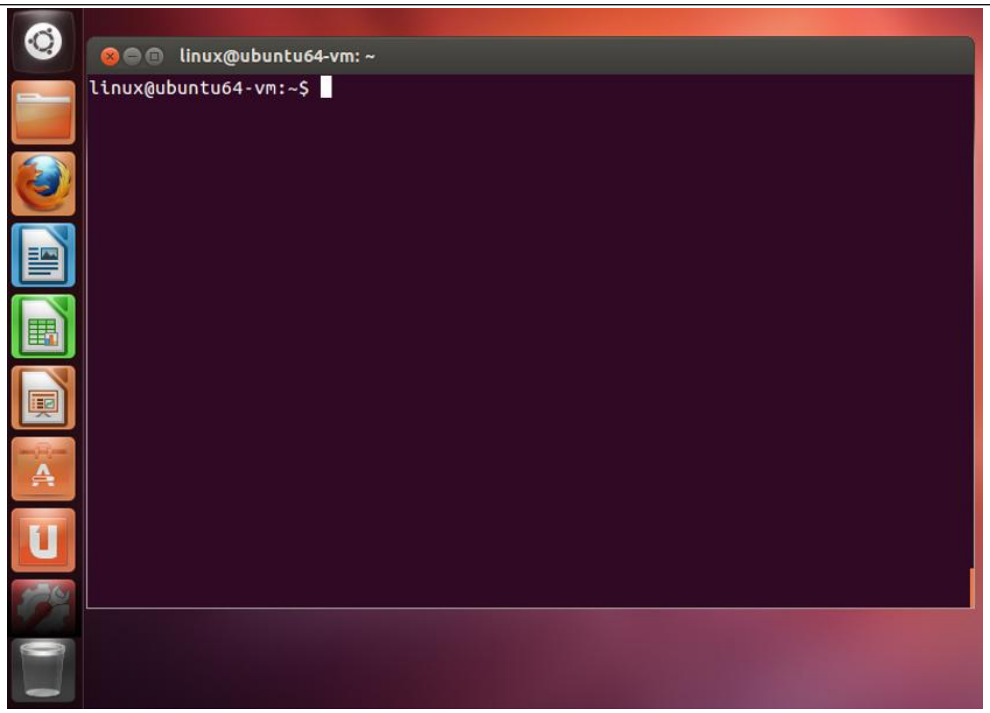
```
[    2.925000] devtmpfs: mounted
[    2.925000] Freeing unused kernel memory: 228K (c0530000 - c0569000)
[root@farsight ]# cd /app
[root@farsight app]# ls
hello
[root@farsight app]# ./hello
hello,world!
[root@farsight app]#
```

C++ Code

```
1   #include <stdio.h>
2
3 int main (int argc, char **argv)
4 {
5     printf("hello,world!\n");
6
7     return 0;
8 }
9
```

4.3 第 7 章　Linux 系统 Makefile 编写实验

4.3.1 准备环境



4.3.2 拷贝代码

将【华清远见-CORTEXA9 资料：实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\05. Linux 系统 Makefile 编写实验/实验代码/5.2/makefileTest】目录拷贝到虚拟机共享目录下。

将 Makefile 部分代码拷贝到虚拟机 Linux 下。

建立相关目录；

```
$ cd workdir/linux/application

$ mkdir 7-Makefile
```

```
linux@ubuntu64-vm:~/workdir/linux/application$ mkdir 7-Makefile
linux@ubuntu64-vm:~/workdir/linux/application$ ls
5-shell  6-gcc  7-Makefile
```

将代码从共享目录拷入虚拟机 Linux 操作系统下；

```
$ cp /mnt/hgfs/share/实验代码/05.\ Linux 系统 Makefile 编写实验/实验代码
/5.2/makefileTest/* 7-Makefile/ -a

$ cd 7-Makefile/
```

将代码从共享目录拷入虚拟机 Linux 操作系统下；

```
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/实验代码/05.\
Linux系统Makefile编写实验/实验代码/5.2/makefileTest/* 7-Makefile/ -a
linux@ubuntu64-vm:~/workdir/linux/application$ ls
5-shell  6-gcc  7-Makefile
linux@ubuntu64-vm:~/workdir/linux/application$ cd 7-Makefile/
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ ls
bin  f1  f2  include  main  Makefile  obj
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$
```

4.3.3 执行代码

进入 makefileTest 目录，执行 make。

```
$ make CLEAN

$ make
```

代码如下：

```
CC = arm-none-linux-gnueabi-gcc
SUBDIRS = f1 \
          f2 \
          main \
          obj

OBJS = f1.o f2.o main.o
BIN = myapp
OBJS_DIR = obj
BIN_DIR = bin
export CC OBJS BIN OBJS_DIR BIN_DIR

all : CHECK_DIR $(SUBDIRS)
    cp bin/myapp /source/rootfs/app
CHECK_DIR :
    mkdir -p $(BIN_DIR)
$(SUBDIRS) : ECHO
    make -C $@
ECHO:
    @echo $(SUBDIRS)
    @echo begin compile
CLEAN :
    @$(RM) $(OBJS_DIR)/*.o
    @rm -rf $(BIN_DIR)
```

执行结果如下：

```
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ make CLEAN
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ make
mkdir -p bin
f1 f2 main obj
begin compile
make -C f1
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/f1'
arm-none-linux-gnueabi-gcc -c f1.c -o ../obj/f1.o
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/f1'
make -C f2
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/f2'
arm-none-linux-gnueabi-gcc -c f2.c -o ../obj/f2.o
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/f2'
make -C main
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/main
arm-none-linux-gnueabi-gcc -c main.c -o ../obj/main.o
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/main'
make -C obj
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/obj'
arm-none-linux-gnueabi-gcc -o ../bin/myapp f1.o f2.o main.o
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/obj'
cp bin/myapp /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$
```

目录树结构如下：

```
.
|-- Makefile
|-- bin
|   `-- myapp
|-- f1
|   |-- Makefile
|   `-- f1.c
|-- f2
|   |-- Makefile
|   `-- f2.c
|-- include
|   `-- myinclude.h
|-- main
|   |-- Makefile
|   `-- main.c
`-- obj
    |-- Makefile
    |-- f1.o
    |-- f2.o
    `-- main.o

6 directories, 13 files
```

我们看到在 bin 目录下生成了我们的目标文件 myapp，并且拷贝到了/source/rootfs/app 下，在 obj 目录下生成了.o 的中间文件。

```
linux@ubuntu64-vm:~$ cd /source/rootfs/app
linux@ubuntu64-vm:/source/rootfs/app$ ls
myapp
```

按照 3.4.10 章节的步骤，通过 tftp 下载内核，nfs 挂载文件系统，启动开发板。

```
[    1.585000] drivers/rtc/hctosys.c: unable to open rtc device (rtc0)
[    1.600000]  mmcblk0boot1: unknown partition table
[    1.605000] dm9000 5000000.ethernet eth0: link down
[    1.610000]  mmcblk0boot0: unknown partition table
[    1.785000] mmc0: new high speed SDHC card at address aaaa
[    1.790000] mmcblk1: mmc0:aaaa SS08G 7.40 GiB
[    1.805000]  mmcblk1: p1
[    1.810000] IP-Config: Guessing netmask 255.255.255.0
[    1.810000] IP-Config: Complete:
[    1.815000] dm9000 5000000.ethernet eth0: link up, 10Mbps, full-duplex, lpa 0
x4C61
[    1.820000]      device=eth0, hwaddr=00:0a:2d:a6:55:a2, ipaddr=192.168.100.19
1, mask=255.255.255.0, gw=255.255.255.255
[    1.835000]      host=192.168.100.191, domain=, nis-domain=(none)
[    1.840000]      bootserver=255.255.255.255, rootserver=192.168.100.192, root
path=
[    1.845000] clk: Not disabling unused clocks
[    1.890000] usb 1-3: new high-speed USB device number 2 using exynos-ehci
[    2.025000] hub 1-3:1.0: USB hub found
[    2.025000] hub 1-3:1.0: 3 ports detected
[    2.855000] VFS: Mounted root (nfs filesystem) on device 0:10.
[    2.860000] devtmpfs: mounted
[    2.865000] Freeing unused kernel memory: 228K (c0530000 - c0569000)
[root@farsight ]#
```

在开发板串口终端执行应用程序。

```
# cd app

# ./myapp
```

```
[    1.785000] mmc0: new high speed SDHC card at address aaaa
[    1.790000] mmcblk1: mmc0:aaaa SS08G 7.40 GiB
[    1.805000]  mmcblk1: p1
[    1.810000] IP-Config: Guessing netmask 255.255.255.0
[    1.810000] IP-Config: Complete:
[    1.815000] dm9000 5000000.ethernet eth0: link up, 10Mbps, full-duplex, lpa 0
x4C61
[    1.820000]      device=eth0, hwaddr=00:0a:2d:a6:55:a2, ipaddr=192.168.100.19
1, mask=255.255.255.0, gw=255.255.255.255
[    1.835000]      host=192.168.100.191, domain=, nis-domain=(none)
[    1.840000]      bootserver=255.255.255.255, rootserver=192.168.100.192, root
path=
[    1.845000] clk: Not disabling unused clocks
[    1.890000] usb 1-3: new high-speed USB device number 2 using exynos-ehci
[    2.025000] hub 1-3:1.0: USB hub found
[    2.025000] hub 1-3:1.0: 3 ports detected
[    2.855000] VFS: Mounted root (nfs filesystem) on device 0:10.
[    2.860000] devtmpfs: mounted
[    2.865000] Freeing unused kernel memory: 228K (c0530000 - c0569000)
[root@farsight ]# cd app
[root@farsight app]# ./myapp
Message from f1.c...
Message from f2.c...
[root@farsight app]#
```

我们也可以用如下命令清除中间文件和目标文件，恢复 make 之前的状态，在虚拟机下输入：

$ make CLEAN

```
.
|-- Makefile
|-- f1
|   |-- Makefile
|   `-- f1.c
|-- f2
|   |-- Makefile
|   `-- f2.c
|-- include
|   `-- myinclude.h
|-- main
|   |-- Makefile
|   `-- main.c
`-- obj
    `-- Makefile

5 directories, 9 files
```

我们可以看到已经变为 make 之前的目录状态了。

五、实验数据（现象）处理分析

5.1 第 5 章实验 Linux Shell 编程

```
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$ chmod 777 file_or_dir.sh
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$ ./file_or_dir.sh
case_and_if.sh is a file
file_or_dir.sh is a file
simple is a directory
unload.sh is a file
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$
```

5.2 第 6 章 GCC 编译实验

成功输出：

```
[    2.925000] devtmpfs: mounted
[    2.925000] Freeing unused kernel memory: 228K (c0530000 - c0569000)
[root@farsight ]# cd /app
[root@farsight app]# ls
hello
[root@farsight app]# ./hello
hello,world!
[root@farsight app]#
```

5.3 第 7 章　Linux 系统 Makefile 编写实验

在 make 之前，树形结构为如下图所示：

```
.
|-- Makefile
|-- f1
|   |-- Makefile
|   `-- f1.c
|-- f2
|   |-- Makefile
|   `-- f2.c
|-- include
|   `-- myinclude.h
|-- main
|   |-- Makefile
|   `-- main.c
`-- obj
    `-- Makefile

5 directories, 9 files
```

进行 make 以后，树形结构也进行了变化，多了几个编译产生的文件，如下图所示：

```
.
|-- Makefile
|-- bin
|   `-- myapp
|-- f1
|   |-- Makefile
|   `-- f1.c
|-- f2
|   |-- Makefile
|   `-- f2.c
|-- include
|   `-- myinclude.h
|-- main
|   |-- Makefile
|   `-- main.c
`-- obj
    |-- Makefile
    |-- f1.o
    |-- f2.o
    `-- main.o

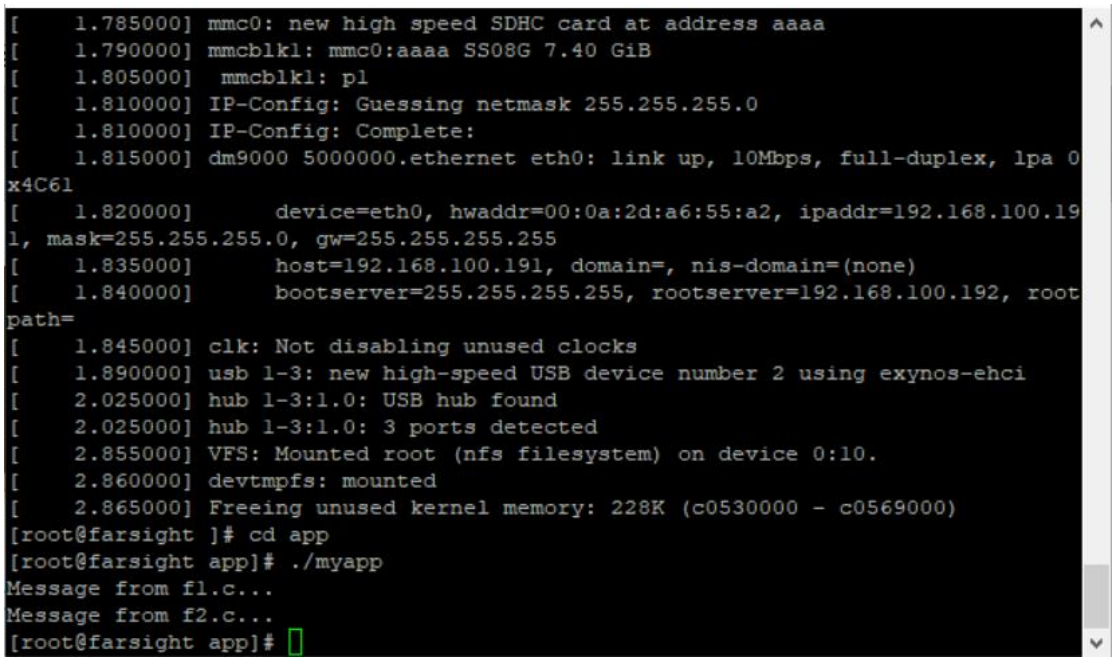
6 directories, 13 files
```

在终端目录输入 ls 查看都有什么文件，如下图所示：

```
[root@farsight ]# ls
app     dev     lib     mnt     proc    sbin    tmp     var
bin     etc     linuxrc myapp   root    sys     usr     ??????
[root@farsight ]# cd app
```

发现 app 和 myapp 文件也是被拷贝到此，所以我们上述的实验步骤都是完全正确的。

最后进入到 app 目录里面，输入./myapp 命令，查看输出结果（实验现象）：



可以看到信息都是来自保存到目录下面的 f1.c 和 f2.c 文件，与实验预期结果一样。

六、实验结论：

通过本次实验，我深入理解了 Linux Shell 编程 的基本概念和操作方式，掌握了如何编写简单的 Shell 脚本 来实现自动化任务。同时，我学习了 Linux 系统中的多种 Shell 版本及其特点，如 Bash、C Shell、Z Shell 等，进一步提高了对 Linux 命令行环境的熟悉度。

在 GCC 编译实验 部分，我掌握了 GCC 的基本工作流程，包括 预处理、编译、汇编和链接 四个阶段。实验中，我使用不同的 GCC 选项，如 -c 进行单独编译，-o 指定输出文件，以及 -I 和 -L 选项来管理头文件和库文件的路径。这些知识对于 嵌入式系统开发和跨平台编译 具有重要意义。

在 Makefile 编写实验 中，我学习了 Makefile 的基本结构、依赖管理和变量定义方式。通过实验，我能够编写高效、可维护的 Makefile 文件，利用 Make 工具 自动化编译流程，减少手动输入命令的重复性，提高项目管理的效率。这对于大型软件开发和嵌入式系统编译具有实际应用价值。

综合来看，本次实验帮助我提升了 Linux 编程、GCC 编译和 Makefile 管理 方面的能力，为后续的嵌入式系统开发奠定了坚实的基础。

七、指导教师批阅意见： **成绩评定：** 指导教师签字： 年　　月　　日
八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。